



ADS06: 回溯 (Backtracking)

来源: /home/vincent/Code/ads/Materials/Student Slides/ADS06Backtracking_Stu.ppt

课件范围: S1-S2、S3-S6、S7-S9、S10-S11、S12-S17、S18-S19; 共 19 张幻灯片。

正文按原课件顺序整理; S 编号指幻灯片顺序。关键公式的补充与原文疑点均明确标注。

本讲路线图 (Lecture Roadmap)

穷举与剪枝 → 八皇后及隐式搜索树 → 收费公路重建 → 回溯模板与变量顺序 → 井字棋的极小极大策略与 α - β 剪枝 → 项目建模。

1. 回溯的基本思想 (Rationale of Backtracking) [S1-S2]

回溯 (Backtracking) 逐步构造候选解, 在部分解已经违反约束时停止向下扩展, 从而跳过整片不可能产生答案的搜索空间。[S1-S2]

重要性: A · 必须掌握

设完整解为 $(x_1, \dots, x_n)$, 其中 $x_i \in S_i$ 。已经确定前 i 个变量时, 按以下情况处理:

1. **存在尚未尝试的候选:** 从 S_{i+1} 中选择一个值, 检查新的部分解。
2. **新部分解满足必要约束:** 接受这个选择, 继续确定下一变量。
3. **新部分解违反约束:** 撤销这个选择, 尝试同一层的下一个候选。
4. **本层候选全部失败:** 返回上一层, 撤销上一层的选择, 再尝试它的其他候选。
5. **已确定全部变量:** 得到完整解, 按需求返回一个解或记录后继续枚举。[S2、S10]

剪枝 (Pruning) 成立的条件是: 被否定的部分解不可能扩展成合法完整解。搜索树有限、候选枚举完整、剪枝判断可靠时, 运行到结束就能找到存在的答案, 或确认无解。[S2]

核心原理 (Core Principle)

一旦当前前缀中出现无法由后续选择消除的冲突, 它的所有扩展都保留这个冲突, 因此可以一起舍弃。

1. 搜索树 (Search Tree) 的根表示空解, 第 i 层表示前 i 个变量已经赋值。
2. 每条边代表一次候选选择, 每条根到叶的路径对应一个完整候选。
3. 只在满足必要约束的节点继续搜索, 实际执行的是带剪枝的深度优先搜索 (Depth-First Search, DFS)。
4. 回退时必须恢复现场, 使下一分支看到与进入本层前相同的状态。[S2、S5-S6、S10]

解释补充： 一个候选失败时应先尝试本层其他候选；只有本层全部失败才返回更上一层。剪枝不能仅凭“目前看起来不够好”就否定仍有可能完成的解。

2. 八皇后与隐式搜索树 (Eight Queens and Implicit Search Tree) [S3-S6]

用“每行一个皇后”的变量设计直接排除同行冲突，再用列与对角线约束筛选合法排列。[S3-S4]

2.1 解的表示与约束 (Representation and Constraints) [S3-S4]

重要性：A · 必须掌握

在 8×8 棋盘上放置 8 个皇后，任何两个皇后不能处在同一行、同一列、主对角线或副对角线上。令 Q_i 是第 i 行的皇后， x_i 是其列号，则解为 $(x_1, \dots, x_8)$ 。[S3-S4]

约束分成三类：

1. **取值范围：** $x_i \in \{1, 2, \dots, 8\}$ 。
2. **列约束：** $i \neq j$ 时， $x_i \neq x_j$ 。
3. **对角线约束：** $i \neq j$ 时， $|x_i - x_j| \neq |i - j|$ ，等价于 $(x_i - x_j)/(i - j) \neq \pm 1$ 。[S4]

候选空间也应分层理解：

1. 只规定每行选择一列时，有 8^8 个列号向量。
2. 再排除重复列时，候选是 $1, \dots, 8$ 的排列，共 $8!$ 个。
3. 加入对角线约束后，才得到真正的八皇后解。一般 n 皇后的排列候选数为 $n!$ ，它不是合法解数。[S4]

一个给定解为 $(4, 6, 8, 2, 7, 1, 3, 5)$ 。检查时可分别验证列号互异、各 $i - x_i$ 互异、各 $i + x_i$ 互异；后两组分别对应两种对角线。[S4]

解释补充： 同一条主对角线上的行列差固定，同一条副对角线上的行列和固定，因此可用列、行列差、行列和三个集合做增量冲突检查。集合只记录已放置的皇后，回退时也必须删除相应记录。

2.2 四皇后的搜索过程 (Four-Queens Search) [S5-S6]

重要性：B · 重点理解

四皇后的排列搜索树有 $4!$ 个叶节点；实际算法不需要建立或保存整棵树，只保存当前路径与恢复状态所需的信息。[S5-S6]

核心原理 (Core Principle)

每放置一个皇后，只检查它与前面皇后的冲突；失败的前缀立即回退，成功的前缀才继续增长。

以下是按列号递增尝试时的一段搜索说明，最终得到 $(2, 4, 1, 3)$ ：

1. 先尝试 $x_1 = 1$ 。第二行的第 1 列冲突，第 2 列有对角线冲突；尝试第 3 列后，第三行没有合法列，因而回退。
2. 在前缀 (1, 4) 下，第三行只能尝试第 2 列，但第四行仍无合法列。因此以 $x_1 = 1$ 开头的分支全部失败。
3. 改为 $x_1 = 2$ ；第二行尝试到 $x_2 = 4$ 时不冲突。
4. 第三行取 $x_3 = 1$ ，第四行取 $x_4 = 3$ ，得到合法解 (2, 4, 1, 3)。[S5-S6]

解释补充： 上述按候选顺序的文字轨迹用于解释剪枝机制，没有复刻搜索图中的动画或节点编号。另一解是 (3, 1, 4, 2)，可由棋盘左右反射得到；S6 将整个过程称为深度优先搜索，不能据此把实现限定为必须显式建树的“后序遍历”。

3. 收费公路重建 (Turnpike Reconstruction) [S7-S9]

根据所有点对距离恢复直线上的点坐标。这个问题展示了“候选位置只有两种，但局部合法仍可能导致后续失败”的回溯过程。[S7-S9]

3.1 问题定义与端点固定 (Problem and Endpoints) [S7]

重要性： A · 必须掌握

给定 $N(N-1)/2$ 个点对距离，恢复满足

$$0 = x_1 < x_2 < \cdots < x_N$$

的点集。距离必须按多重集 (Multiset) 处理，同一个数出现几次，就代表几对点产生该距离。[S7]

对于

$$D = \{1, 2, 2, 2, 3, 3, 3, 4, 5, 5, 5, 6, 7, 8, 10\},$$

共有 15 个距离，故 $N(N-1)/2 = 15$ ，得到 $N = 6$ 。最大距离 10 是两个端点的距离，因此固定 $x_1 = 0, x_6 = 10$ ，从 D 中删除一次 10。[S7]

3.2 最大剩余距离的两个分支 (Two Branches from the Largest Distance) [S7-S9]

重要性： A · 必须掌握

设总长度 $L = x_N$ ，当前最大剩余距离为 d 。按原算法的顺序尝试：

1. **右侧候选：** 将当前最右未定点放在 d ，即 $x[\text{right}] = d$ 。
2. **左侧候选：** 右侧候选失败后，将当前最左未定点放在 $L - d$ ，即 $x[\text{left}] = L - d$ 。[S8-S9]

每次尝试位置 y ，必须检查到所有已确定点的距离多重集

$$R_y = \{|y - x| : x \text{ 是已确定点}\}$$

能否从剩余 D 中逐项扣除。若某个距离需要两次而 D 只剩一次，检查必须失败。[S8-S9]

核心原理 (Core Principle)

已确定点之间的距离已经删除；剩余最大距离必能由某个未确定点到左端点或右端点产生，所以只需尝试两个镜像位置。

补充推导： 设未确定点的最小、最大坐标分别为 u, v 。任一含未确定点的距离，都不大于 $\max(v, L - u)$ ；而 $v - 0$ 与 $L - u$ 都仍在剩余距离中。因此 $d = \max(v, L - u)$ ，对应 $v = d$ 或 $u = L - d$ 。成立前提是每步都精确删除已确定点对的距离，且所有点位于 $[0, L]$ 内。

局部检查通过时删除 R_y 并递归；递归失败时原样补回 R_y 。只有多重集与已确定点都恢复一致，第二个分支才有正确的输入。[S8-S9]

3.3 六点示例与撤销操作 (Six-Point Example and Undo) [S7-S9]

重要性：B · 重点理解

核心原理 (Core Principle)

先接受一个局部合法的候选，再用后续失败否定该分支；撤销时恢复距离的重数，最终得到与全部输入距离一致的点集。

下面按 S8-S9 的候选顺序，用 S7 的数据补全计算：

1. 已知 $\{0, 10\}$ ，最大剩余距离为 8。放置 8，需要删去 $\{8, 2\}$ ，检查通过。
2. 最大剩余距离变为 7。先放置 7，需要删去 $\{7, 3, 1\}$ ，也通过。
3. 现在最大剩余距离为 6。放置 6 需要 $\{6, 4, 2, 1\}$ ，但唯一的 1 已用掉，失败；放置 $10 - 6 = 4$ 需要 $\{4, 6, 4, 3\}$ ，但 4 只有一份，也失败。
4. 撤销点 7，补回 $\{7, 3, 1\}$ 。改为放置 $10 - 7 = 3$ ，删去到 0, 10, 8 的距离 $\{3, 7, 5\}$ 。
5. 放置 6，删去到 0, 10, 8, 3 的距离 $\{6, 4, 2, 3\}$ 。
6. 放置 5，删去到 0, 10, 8, 3, 6 的距离 $\{5, 5, 3, 2, 1\}$ 。剩余多重集为空，得到 $(0, 3, 5, 6, 8, 10)$ 。[S7-S9]

补充推导： 以上逐次扣除和回滚是对给定距离例子的计算展开。最终按各左端点列出距离：从 0 出发为 3, 5, 6, 8, 10；从 3 出发为 2, 3, 5, 7；从 5 出发为 1, 3, 5；从 6 出发为 2, 4；从 8 出发为 2。合计 15 项，排序后与输入多重集完全一致。

3.4 递归接口与不变量 (Recursive Interface and Invariant) [S8-S9]

重要性：A · 必须掌握

$\text{Reconstruct}(X, D, N, \text{left}, \text{right})$ 的状态按以下规则解释：

1. $X[1..left-1]$ 已确定, 位于左侧。
2. $X[right+1..N]$ 已确定, 位于右侧。
3. $X[left..right]$ 尚未确定。
4. D 只保留至少涉及一个未确定点的点对距离。[S8–S9]

保留原分支顺序的等价伪代码如下; $required(y)$ 按重数计算到当前已知点的距离:

```
Reconstruct(left, right):
    if D is empty:
        return true
    d = maximum remaining distance in D

    for (position, y) in [(right, d), (left, L-d)]:
        R = required(y)
        if y is a valid new position and D contains multiset R:
            X[position] = y
            remove multiset R from D
            if position == right:
                found = Reconstruct(left, right-1)
            else:
                found = Reconstruct(left+1, right)
            if found:
                return true
            restore multiset R to D
            clear this choice
    return false
```

解释补充: 这是原算法的结构化表达。实现还应核验坐标不重复、位于未定区间内, 以及成功时确已放满 N 个点; 在合法输入及不变量成立时, 剩余距离为空的原始终止条件与完整重建一致。左右候选相同可避免重复搜索, 但不能因此漏掉有效分支。

4. 通用模板与搜索顺序 (Template and Search Order) [S10–S11]

回溯效率由候选规模、约束检查代价和实际通过检查的节点数共同决定。[S10–S11]

4.1 求一个解的模板 (Template for One Solution) [S10]

重要性：A · 必须掌握

```
Backtracking(i):
    if i > N:
        return true
    for each candidate x in S[i]:
        if Check(current prefix extended by x):
            apply x
            if Backtracking(i+1):
                return true
            undo x
    return false
```

模板的分支分类：

1. **检查失败：** 不进入递归，也不保留本候选的状态改变。
2. **检查成功、递归失败：** 执行 Undo，继续尝试本层候选。
3. **递归成功：** 返回 true 并保留完整解；原模板会停止枚举。[S10]

解释补充： 要枚举全部解，应在完整解处记录答案，再继续搜索，并确保每条成功分支退出时也恢复状态。不能直接沿用“找到第一个解就退出”的模板。

4.2 复杂度与小集合优先 (Cost and Smaller Domain First) [S10–S11]

重要性：B · 重点理解

候选集合大小不同时，优先安排较小的 S_i ，通常能更早暴露冲突，减少搜索树宽度。[S11]

按开销分类：

1. **候选越多：** 无剪枝时完整候选数为 $\prod_i |S_i|$ ，还要付出访问中间节点的代价。
2. **约束检查越复杂：** 单节点代价越高。更强剪枝若检查本身太贵，总时间未必改善。
3. **通过检查的节点越多：** 向下递归越多；这一数量依赖实例，往往难以精确估计。[S10 的备注]

解释补充： 若实际检查了 V_s 个节点，每个节点检查、修改和撤销的总成本至多为 C ，总时间可写为 $O(V_s C)$ 。对 n 皇后的排列枚举，候选叶数是 $n!$ ；直接逐个比较已有皇后的检查会再产生额外因子，不能把 $n!$ 不加条件地当成所有实现的精确运行时间。小集合优先是搜索启发式，重新排序变量时也必须同步调整约束和答案的映射。

5. 对抗游戏与剪枝 (Adversarial Games and Pruning) [S12–S17]

井字棋 (Tic-Tac-Toe) 中的双方目标相反，搜索值需要在“取最大”和“取最小”之间交替回传。[S12–S15]

5.1 井字棋状态与评价函数 (States and Evaluation Function) [S12–S14]

重要性：B · 重点理解

一方在横向、纵向或对角线上连成三个自己的标记即获胜。[S12]

状态规模需区分两种计数：

1. **棋盘填充方式**：九个格子各可为 X、O 或空，得到 $3^9 = 19\,683$ 种组合。
2. **填满格子的顺序**：九个位置的排列为 $9! = 362\,880$ 种。[S13]

解释补充：这些是宽松的组合计数；轮流落子、双方棋子数关系和提前结束会排除大量不合法或不可达状态。它们不等于精确的合法井字棋局数。

评价函数 (Evaluation Function) 量化局面 P 对计算机的有利程度。记 W 为仍可能形成胜利的线路数，X 为计算机，O 为人类；计算机希望评价值尽量大，人类希望它尽量小。[S14 及备注]

解释补充：一种与这段描述一致的常见示例是 $E(P) = W_X(P) - W_O(P)$ ，其中 W_X 统计不含 O 的胜利线路， W_O 统计不含 X 的胜利线路。这是用来说明评分方向的补充定义；S14 的具体公式在提取与定点渲染中均不可读，不能认定它就是原公式。终局胜负应有优先于普通启发式分数的明确取值。

5.2 极小极大策略 (Minimax Strategy) [S14–S15]

重要性：A · 必须掌握

极小极大 (Minimax) 假设双方都按自身目标选择最优后续行动：

1. **轮到计算机的 MAX 节点**：取所有后继值的最大值。
2. **轮到人类的 MIN 节点**：取所有后继值的最小值。
3. **终局或规定搜索深度**：返回终局效用或评价函数值。[S14–S15]

核心原理 (Core Principle)

对手不会替自己选择有利的分支，因此在自己的回合取最大值，在对手的回合必须按最小值估计。

用 $V(P)$ 表示回传值，有

$$V(P) = \begin{cases} E(P), & P \text{ 是叶节点,} \\ \max_{Q \in \text{Children}(P)} V(Q), & P \text{ 是 MAX 节点,} \\ \min_{Q \in \text{Children}(P)} V(Q), & P \text{ 是 MIN 节点.} \end{cases}$$

补充推导：该递推补全了 S15 的自底向上取值规则。例如两个 MIN 子树的叶分数分别是 $\{1, 2\}$ 和 $\{-1, 0\}$ ，回传值是 1 和 -1 ，其 MAX 父节点取 1。这是用于说明规则的数值例子，不是对原搜索图节

点的逐一复刻。搜索到全部终局时给出精确的 minimax 结果；截断搜索得到的是依赖评价函数的估计。

5.3 α 剪枝与 β 剪枝 (Alpha and Beta Cutoffs) [S16–S17]

重要性：A · 必须掌握

剪枝依据已经搜索到的界，而不是猜测未搜索分支的具体值。

1. **α 剪枝**：MAX 父节点已有值 44。另一个 MIN 子节点已经看到一个后继值 40，则这个 MIN 节点的最终值至多为 40。MAX 已有更好的 44，因此该 MIN 节点剩余后继可以不搜。[S16]
2. **β 剪枝**：MIN 父节点已有值 44。另一个 MAX 子节点已经看到一个后继值 68，则这个 MAX 节点的最终值至少为 68。MIN 已有更小的 44，因此该 MAX 节点剩余后继可以不搜。[S17]
3. **α - β 剪枝 (Alpha-Beta Pruning)**：在同一次递归搜索中同时维护两种界。[S17]

核心原理 (Core Principle)

当一个分支即使取到它仍可能达到的最好结果，也无法改变祖先已有的选择时，继续搜索这个分支不会改变根的 minimax 值。

补充推导：用 α 表示 MAX 在当前路径上已有的保证下界， β 表示 MIN 已有的保证上界。MAX 节点逐步更新 $\alpha = \max(\alpha, v)$ ；MIN 节点逐步更新 $\beta = \min(\beta, v)$ 。一旦 $\alpha \geq \beta$ 即可截断。等号时根的最优值不变，但若枚举全部并列最优策略，应另行处理被略去的等值分支。

复杂度按搜索顺序分类：

1. **差的落子顺序**：可能几乎不剪枝，仍访问 $O(b^d)$ 个节点，其中 b 是分支因子， d 是搜索深度。
2. **理想顺序、均匀树的经典估计**：可降到约 $O(b^{d/2})$ ，相当于完整树规模的平方根量级。
3. **一般实际局面**：效果依赖树形、评分和落子顺序；不是无条件保证访问 $\sqrt{N}$ 个节点。[S17]

解释补充：S17 的具体节点数量公式无法可靠恢复，上述理想与最坏界是对 α - β 搜索的条件化说明。S12 的 AlphaGo 标题只作为游戏搜索引入，不能把这里的井字棋 minimax 模板当作 AlphaGo 完整算法。

6. 安全水果项目与阅读 (Safe Fruit Project and Reading) [S18–S19]

6.1 项目建模 (Project Modeling) [S18]

重要性：C · 了解即可

给出一些不能一起选择的水果组合，从一篮水果中挑出互相兼容的水果。题目完整目标、输入规模、约束及输出规则以 Pintia 上的项目说明为准；S18 未提供这些细节。[S18]

解释补充： 可将水果建模为顶点，禁配组合建模为无向边。合法选择中不能同时包含一条边的两个端点，因此对应图的独立集（Independent Set）。若完整题目要求尽量多选，才进一步成为最大独立集问题；不能仅凭“安全地选出水果”就假定最大化目标。这里的禁配关系是算法题输入，不是医学或营养结论。

回溯建模的基本分类是：对当前水果尝试“选择”和“不选择”；选择时检查与已选集合是否冲突，失败则剪枝；记录答案的方式取决于完整题目要求。[S18；建模为解释补充]

6.2 参考阅读（Further Reading）[S19]

《Data Structure and Algorithm Analysis in C》第 2 版第 10 章，课件列出页码 403–414；不同版本页码可能不同。[S19]

本讲总结（Lecture Summary）

- 回溯用必要约束否定部分解，一次跳过整片无效子树；撤销操作是正确性的组成部分。[S2、S10]
- 皇后问题的行约束由变量表示保证，列约束把候选缩成排列，对角线约束进一步筛选。[S3–S6]
- 收费公路重建必须维护距离重数；最大剩余距离只产生两个候选位置。[S7–S9]
- 通用模板默认找到一个解即退出；枚举所有解要调整成功后的控制流。[S10]
- 小候选集合优先通常有利于尽早剪枝，但运行时间仍依赖实际访问节点数和检查代价。[S10–S11]
- Minimax 交替取最大、最小； α - β 通过上下界舍弃不会改变根值的分支。[S14–S17]

复杂度与适用条件速查（Complexity and Conditions）

方法或计数	结果	条件与限制	来源
n 皇后列号向量	n^n 个候选	仅每行选一列	S4
n 皇后排列候选	$n!$ 个候选	列号互异，尚未满足对角线约束	S4–S5
通用回溯	$O(V_s C)$	V_s 为实际检查节点数， C 为每节点代价；补充表达	S10
递归栈与当前解	通常 $O(n)$	深度为 n ，不计输入、辅助检查结构及全部输出	S6、S10；补充
收费公路重建	搜索可能为指数级	每层至多两种位置；距离容器与检查影响单层代价	S8–S9
完整 minimax	$O(b^d)$	分支因子 b 、深度 d 的均匀树估计	S14–S15；补充
α - β 搜索	最坏 $O(b^d)$ ，理想约 $O(b^{d/2})$	理想界需要有利顺序，非每个实例保证	S16–S17；补充

易错点（Common Pitfalls）

1. 把“某个候选失败”当作“整个问题无解”，没有继续尝试同层其他候选。
2. 剪枝否定的条件并非必要条件，误删仍可能完成的解。
3. 回溯只撤销数组值，没有恢复计数、集合或资源状态。
4. 把距离多重集当作普通集合，丢掉重复距离。

5. 把 $n!$ 当作皇后合法解数或所有实现的精确复杂度。
6. 在 MIN 层也取最大值，或把当前子节点界误认为已经精确算出的值。
7. 无条件使用 α - β 的理想复杂度。

复习自测 (Self-Check)

1. 哪类约束可以在完整解产生前安全地剪枝？举出一个不安全剪枝的情形。[S2]
2. 为什么皇后的对角线冲突可写成行差绝对值等于列差绝对值？[S4]
3. 对四皇后，前缀 (1, 4, 2) 为什么不能扩展成解？[S5-S6]
4. 收费公路重建为什么只尝试 d 与 $L - d$ ？为什么要检查距离重数？[S7-S9]
5. 在六点例子中，撤销点 7 时应补回哪三个距离？[S7-S9]
6. 求一个解的模板怎样改为枚举全部解？[S10]
7. 为什么更强的约束检查不一定带来更快的总运行时间？[S10]
8. MAX 已有 44，另一 MIN 分支已有 40 时，为何可剪枝？反过来的 β 剪枝怎样解释？[S16-S17]

术语速查 (Terminology)

中文	English	含义
回溯	Backtracking	构造部分解，失败时恢复并换分支
剪枝	Pruning	排除不可能影响答案的子树
搜索树	Search Tree	用路径表示连续决策
深度优先搜索	Depth-First Search, DFS	沿分支深入，结束后回退
多重集	Multiset	记录元素及其出现次数
收费公路重建	Turnpike Reconstruction	从全部点对距离恢复直线点集
评价函数	Evaluation Function	估计局面对某一方的有利程度
极小极大策略	Minimax Strategy	MAX 与 MIN 交替回传最优值
α - β 剪枝	Alpha-Beta Pruning	根据路径上下界提前截断搜索
独立集	Independent Set	内部不存在相邻顶点的顶点集合

待核对 (Items to Verify)

- [待核对 S14] 评价函数的原始公式在原生文本与定点渲染中均未显示。笔记中的 $W_X - W_O$ 已标为示例性补充；如需逐字对应，请对照教师完整版。
- [待核对 S17] 搜索节点数量的原公式未可靠恢复，不能确认其具体符号或是否附带条件。笔记分别说明理想与最坏界，未把平方根量级作为普遍保证。
- [待核对 S18] Safe Fruit 的完整优化目标、输入约束和输出规则不在本课件中，应以 Pintia 项目说明为准。